CODECELL++ CMPN VESIT

SYRUS HACKATHON 2026

TRACK: **Agentic AI [Rezinix AI]**

PS TITLE: **Autonomous Incident-to-Fix Engineering Agent**

TEAM NAME: **HackHunters**

Problem Understanding

Core Problem

- Debugging production issues is time-consuming
- Logs are often large and unstructured
- Developers must switch between multiple tools
- Root cause identification requires deep system knowledge

Impact

- Increased Mean Time To Resolution (MTTR)
- Downtime affecting users
- Developer productivity loss
- Slower incident response cycles

Opportunity

Analyze logs
Identify root causes
Suggest code fixes
Reduce incident resolution time

Introduction / Proposed Solution

Problem

Production **incident resolution** remains manual and **time-consuming**. Engineers must analyze unstructured logs, trace failures across large codebases, identify runtime/configuration/dependency issues, and **manually** apply and validate fixes.

This results in high **Mean Time To Resoaalution (MTTR)** and reduced developer productivity.

Proposed Solution — FixBot

FixBot is an Agentic AI debugging system that **automates** the **incident-to-resolution** lifecycle. It integrates log analysis, code reasoning, and AI inference to **generate explainable fixes** and structured incident reports.

Core Capabilities

- **Incident Parsing** – Extracts error signals from **logs** and **stack traces**
- **Contextual Code Analysis** – Identifies failure points in the **codebase**
- **Root Cause Inference** – Detects logical, configuration, or **dependency issues**
- **Fix Recommendation** – Generates **minimal**, **targeted** remediation steps
- **Persistent Incident Tracking** – Stores **structured** debugging reports

Workflow Diagram

Production Logs / Error Tickets

↓

Provide Error Log & Code to FixBot

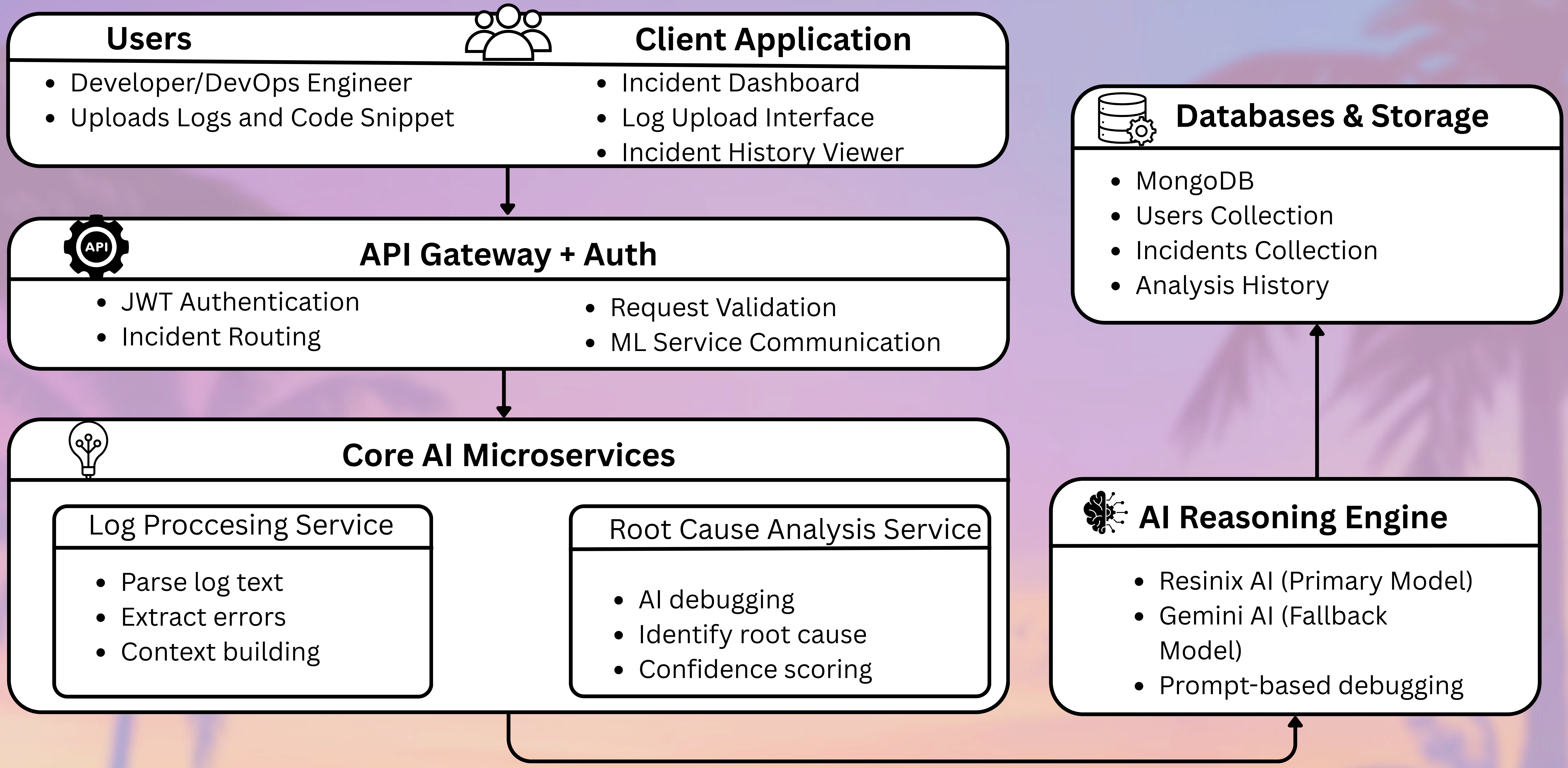
↓

Root Cause + Suggested Fix

↓

Incident Report

Architecture Diagram / Tech stack



Novelty / USP / Showstopper

USP

Incident → Fix Automation

Converts production logs into root cause analysis and fix suggestions.

Dual AI Reliability

Uses Resinix AI (primary) and Gemini (fallback) for continuous AI analysis.

Structured Debugging Reports

Generates severity, root cause, suggested fix, and confidence score.

Persistent Incident History

Stores incidents in MongoDB for tracking and reuse.

Reduced MTTR

AI-powered analysis speeds up debugging and incident resolution.

Implementation Plan

PHASE-1: Setup & Input Handling

- Set up React client, Express server, ML service
- Configure API routes for incident submission
- Implement log + code snippet input handling

PHASE-2: AI Integration

- Build ML inference service
- Integrate Resinix API
- Implement Gemini fallback mechanism

PHASE-3: Analysis Engine

- Design prompt templates for debugging
- Implement log + code context processing
- Generate structured AI analysis output

PHASE-4: Fix Generation Layer

- Process AI responses into fix recommendations
- Format root cause + explanation + confidence
- Standardize incident response schema

PHASE-5: Storage & Reporting

- Implement MongoDB persistence
- Build incident history APIs
- Add email notification & reporting